

GEPA: REFLECTIVE PROMPT EVOLUTION CAN OUTPERFORM REINFORCEMENT LEARNING

Lakshya A Agrawal¹, Shangyin Tan¹, Dilara Soylu², Noah Ziems⁴,
 Rishi Khare¹, Krista Opsahl-Ong⁵, Arnav Singhvi^{2,5}, Herumb Shandilya²,
 Michael J Ryan², Meng Jiang⁴, Christopher Potts², Koushik Sen¹,
 Alexandros G. Dimakis^{1,3}, Ion Stoica¹, Dan Klein¹, Matei Zaharia^{1,5}, Omar Khattab⁶

¹UC Berkeley ²Stanford ³BespokeLabs.ai ⁴Notre Dame ⁵Databricks ⁶MIT

ABSTRACT

Large language models (LLMs) are increasingly adapted to downstream tasks via reinforcement learning (RL) methods like Group Relative Policy Optimization (GRPO), which often require thousands of rollouts to learn new tasks. We argue that the interpretable nature of *language* often provides a much richer learning medium for LLMs, compared to policy gradients derived from sparse, scalar rewards. To test this, we introduce GEPA (Genetic-Pareto), a prompt optimizer that thoroughly incorporates *natural language reflection* to learn high-level rules from trial and error. Given any AI system containing one or more LLM prompts, GEPA samples trajectories (e.g., reasoning, tool calls, and tool outputs) and reflects on them in natural language to diagnose problems, propose and test prompt updates, and combine complementary lessons from the Pareto frontier of its own attempts. As a result of GEPA’s design, it can often turn even just a few rollouts into a large quality gain. Across six tasks, GEPA outperforms GRPO by 6% on average and by up to 20%, while using up to 35x fewer rollouts. GEPA also outperforms the leading prompt optimizer, MIPROv2, by over 10% (e.g., +12% accuracy on AIME-2025), and demonstrates promising results as an inference-time search strategy for code optimization. We release our code at <https://github.com/gepa-ai/gepa>.

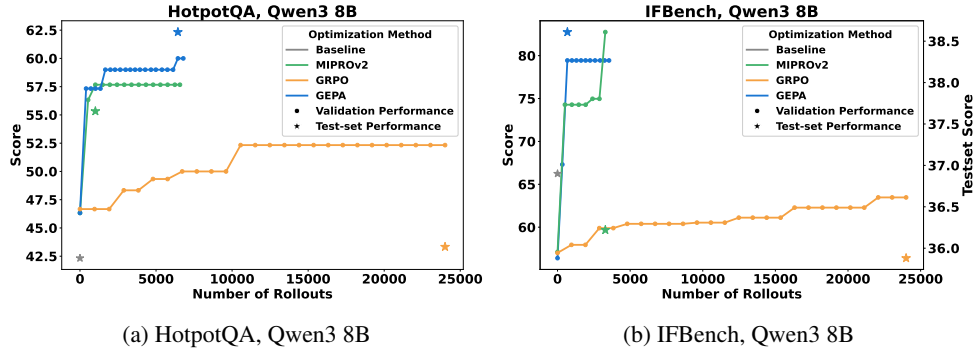


Figure 1: A comparison of learning behavior of the GEPA prompt optimizer against a state-of-the-art prompt optimizer (MIPROv2) and GRPO (24,000 rollouts). As more rollouts are sampled, the prompt optimizers can learn much more quickly than GRPO. GEPA substantially outperforms both GRPO and MIPROv2 in final score. The Test-set star markers demonstrate the performance gap in a held-out set of questions.

1 INTRODUCTION

Large language models (LLMs) have enabled development of agents and systems that combine fuzzy natural-language specifications with tools like retrieval and code execution. This raises the question of how LLMs should be optimized for downstream performance. One popular approach is Reinforcement Learning with Verifiable Rewards (RLVR), e.g. with Group Relative Policy Optimization (GRPO) (Shao et al., 2024), which treats success metrics as end-of-rollout scalar rewards used to estimate policy gradients (Lambert, 2025). While these RL approaches are effective, they typically require tens of thousands of rollouts in practice to fit new tasks. For example, recent works leveraging GRPO typically use up to hundreds of thousands of rollouts for training (Chen et al., 2025b; Wu et al., 2025b; Zhang et al., 2025; Jin et al., 2025; Si et al., 2025; Wang et al., 2025a; Chen et al., 2025a; Sha et al., 2025; Lin et al., 2025a; Peng et al., 2025; Song et al., 2025). This sample inefficiency can quickly become a serious bottleneck: many downstream LLM applications invoke expensive tool calls, have limited inference budget for sampling from the LLM itself, or simply cannot finetune the weights of the largest or best-performing LLMs.

We observe that rollouts sampled from even highly sophisticated LLM systems can be serialized into traces of natural language: they contain nothing but the instructions of each LLM module, the resulting LLM reasoning chains, tool calls, and potentially the internal workings of the reward function (e.g., compiler error messages, before they are collapsed into scalar rewards). Because such serialized trajectories are readily understood by modern LLMs, we argue that *algorithms that learn deliberately in natural language by reflecting on these trajectories* can make more effective use of the strong language priors that LLMs have, compared with standard RL approaches.

We introduce GEPA (Genetic-Pareto), a *reflective* prompt optimizer for compound AI systems that merges textual reflection with multi-objective evolutionary search. GEPA iteratively mutates prompts using natural language feedback drawn from new rollouts. In each mutation, the candidate prompt is derived from an ancestor, accumulating high-level lessons derived from observations and LLM feedback. To avoid local optima that afflict greedy prompt optimization, GEPA maintains a Pareto front: instead of evolving only the global best prompt, it stochastically explores the top-performing prompts for each problem instance. This diversification enables robust generalization and mitigates getting stuck in local minima.

We evaluate GEPA across multi-hop reasoning (HotpotQA; Yang et al. 2018), Math (AIME, LiveBench-Math; Balunović et al. (2025); White et al. (2025)), instruction following (IFBench; Pyatkin et al. 2025b), privacy-aware delegation (PUPA; Li et al. 2025a), and retrieval-augmented verification (HoVer; Jiang et al. 2020), using both open (Qwen3 8B; Yang et al. 2025; Team 2025) and proprietary (GPT-4.1 Mini; OpenAI 2025) models. We find that GEPA generalizes well and is highly sample-efficient: on Qwen3 8B, it outperforms GRPO (24k rollouts) by up to 20% while using up to $35\times$ fewer rollouts, with an average gain of +6% across six tasks. GEPA also surpasses the prior state-of-the-art, MIPROv2 (Opsahl-Ong et al., 2024), on all benchmarks and models, achieving +13% aggregate gains, over double MIPROv2’s +5.6%.

Qualitatively, GEPA-learned prompts are quite rich. Figure 2 shows excerpts from a prompt crafted for the query-creation module of a multi-hop question answering system used in HotpotQA, and Figure 5 shows that even a single *reflective* update often yields large gains. These results highlight that reflective prompt evolution with language feedback enables improved sample efficiency and robust generalization, offering a practical approach to optimizing complex AI workflows in data- or budget-constrained environments. We also demonstrate GEPA as an inference-time search strategy for code optimization on NPUEval (Kalade & Schelle, 2025) & KernelBench (Ouyang et al., 2025) in Sec 5.1, and for adversarial prompt search in Sec 5.2.

2 PROBLEM STATEMENT

We follow related work in defining a **compound AI system** as any modular system composed of one or more language model (LLM) invocations, potentially interleaved with external tool calls, orchestrated through ar-

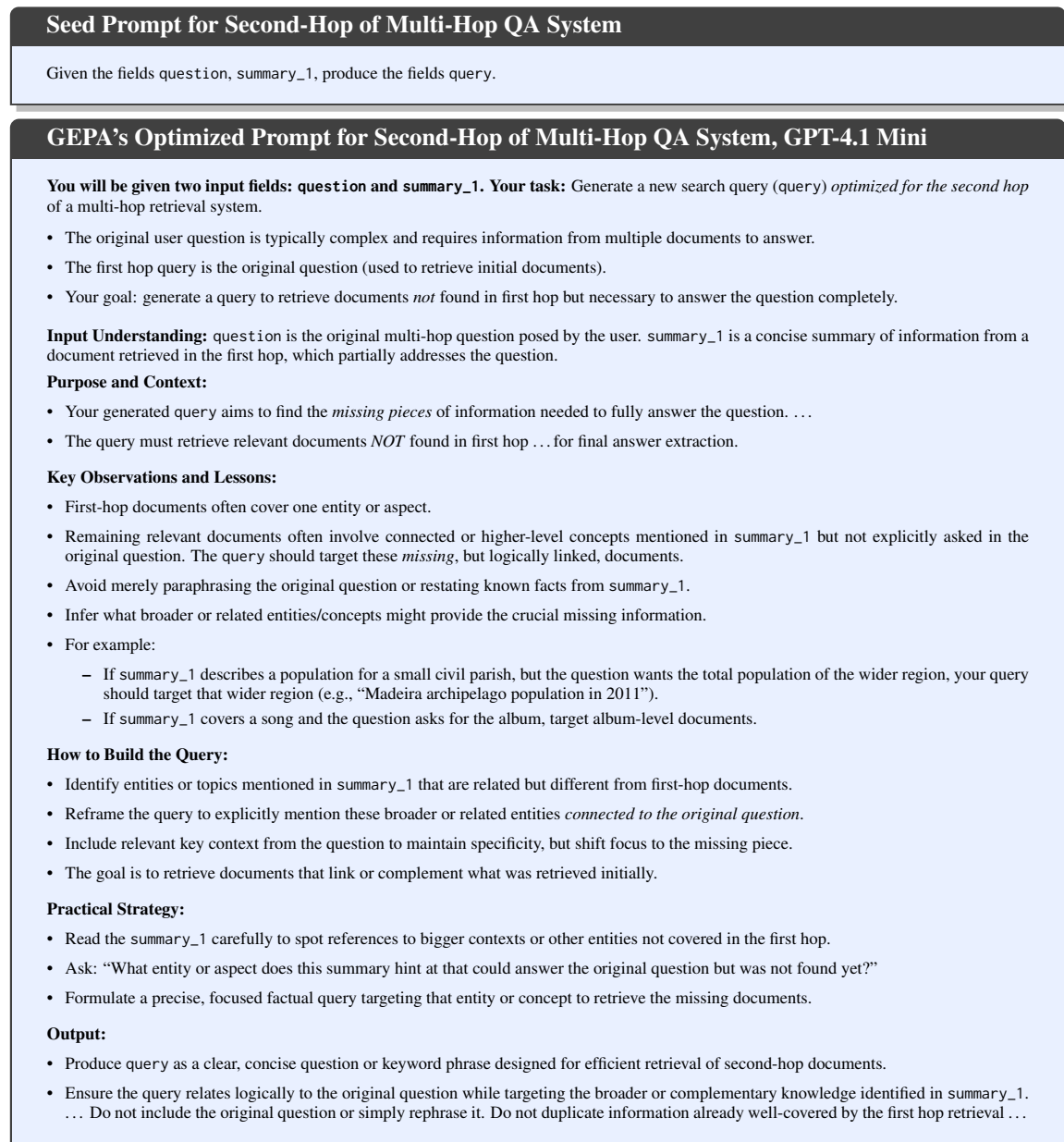


Figure 2: This figure shows an example prompt generated by GEPA for the second-hop document retrieval to be performed in a multi-hop question-answer system, along with the seed prompt it started with. Appendix L compares GEPA’s prompts for all tasks with prompts generated by MIPROv2.

bitrary control flow. This definition subsumes a broad class of real-world LLM-based AI systems, including *agents*, *multi-agent systems*, and general-purpose scaffolding techniques like ReAct (Yao et al., 2023), Ar-

chon (Saad-Falcon et al., 2025), etc. Following Soylu et al. (2024); Khattab et al. (2024); Opsahl-Ong et al. (2024); Tan et al. (2025), we formalize such a system as $\Phi = (M, C, \mathcal{X}, \mathcal{Y})$, where $M = \langle M_1, \dots, M_{|M|} \rangle$ denotes language modules, C specifies control flow logic, and $\mathcal{X}, \mathcal{Y}$ are global input/output schemas. Each module $M_i = (\pi_i, \theta_i, \mathcal{X}_i, \mathcal{Y}_i)$ is an LLM subcomponent: π_i is its (system) prompt including instructions and few-shot demonstrations; θ_i the underlying model weights; $\mathcal{X}_i, \mathcal{Y}_i$ are input/output schemas. At runtime, C orchestrates the sequencing and invocation of modules—e.g., passing outputs from one module to another, invoking modules conditionally, or leveraging tool APIs. This way, C can invoke different modules in any order multiples of times.

Given Φ , let $\Pi_\Phi = \langle \pi_1, \dots, \pi_{|M|} \rangle$ denote the collection of all module prompts and $\Theta_\Phi = \langle \theta_1, \dots, \theta_{|M|} \rangle$ the set of module weights. The learnable parameters are thus $\langle \Pi, \Theta \rangle_\Phi$. For a task instance (x, m) —where x maps to the input schema $\mathcal{X}$ and m contains evaluator metadata (e.g., gold answers, evaluation rubrics, code unit tests)—the system induces an output $y = \Phi(x; \langle \Pi, \Theta \rangle_\Phi)$. A metric $\mu : \mathcal{Y} \times \mathcal{M} \rightarrow [0, 1]$ then measures the output quality of y with respect to metadata m (for example by calculating, exact match, F1, pass rate, etc.). The optimization problem is thus defined as follows, where $\mathcal{T}$ is a task distribution.:

$$\langle \Pi^*, \Theta^* \rangle_\Phi = \arg \max_{\langle \Pi, \Theta \rangle_\Phi} \mathbb{E}_{(x, m) \sim \mathcal{T}} [\mu(\Phi(x; \langle \Pi, \Theta \rangle_\Phi), m)]. \quad (1)$$

We adopt this general problem formulation, allowing updates to both prompts and weights of language modules, to enable comparisons between optimization algorithms that operate in different parameter spaces (e.g., GEPA vs. GRPO).

Sample-Efficient Optimization. In many real-world scenarios, rollouts—concretely, invocations of Φ plus evaluation by μ —are often computationally, monetarily, or timewise expensive. The optimizer is thus limited to at most B rollouts on a dataset $\mathcal{D}_{\text{train}} = \{(x, m)_i\}_{i=1}^N$ with full access to μ . The goal is to identify parameters $\langle \Pi^*, \Theta^* \rangle_\Phi$ that maximize held-out performance, subject to not exceeding the rollout budget B :

$$\langle \Pi^*, \Theta^* \rangle_\Phi = \arg \max_{\langle \Pi, \Theta \rangle_\Phi} \mathbb{E}_{(x, m) \sim \mathcal{T}} [\mu(\Phi(x; \langle \Pi, \Theta \rangle_\Phi), m)], \quad \text{s.t.} \quad \#\text{rollouts} \leq B. \quad (2)$$

The core challenge, then, is: *How do we extract maximal learning signal from every expensive rollout to enable effective adaptation of complex, modular AI systems in low-data or budget-constrained settings?*

3 GEPA: REFLECTIVE PROMPT EVOLUTION

We introduce GEPA (Genetic-Pareto), a sample-efficient optimizer for compound AI systems motivated by three core principles: genetic prompt evolution (Section 3), reflection using natural language feedback (Section 3), and Pareto-based candidate selection (Section 3.1). Figure 3 gives an overview of GEPA and the full GEPA algorithm is formalized in Figure 4. GEPA receives the following inputs: A system Φ instantiated with simple prompts to be optimized, training dataset $\mathcal{D}_{\text{train}}$ (consisting of task instances (x, m) as described in Section 2), the standard evaluation metric μ for the task, a feedback function μ_f (introduced in Section 3) and the total rollout budget B . Note that GEPA evolves only the set of prompts, denoted as Π_Φ , whereas the underlying LLM weights, denoted by Θ_Φ remains fixed.

Genetic Optimization Loop: Given an AI system Φ , the goal is to identify parameters Π_Φ that maximize task performance. GEPA begins with a *candidate pool* $\mathcal{P}$ containing only the base system, where each *candidate* is a concrete instantiation of $\langle \Pi, \Theta_{\text{frozen}} \rangle_\Phi$. It then enters an optimization loop, repeatedly proposing new candidates until the evaluation budget is exhausted. Candidates are derived from existing ones via *reflective mutation* or *crossover*, guided by feedback from rollouts, with each inheriting learning signals from its parents and its own rollout so that GEPA accumulates knowledge along the genetic tree. In each iteration, GEPA (i) selects promising candidates, (ii) proposes and evaluates a variant on a minibatch of tasks, and

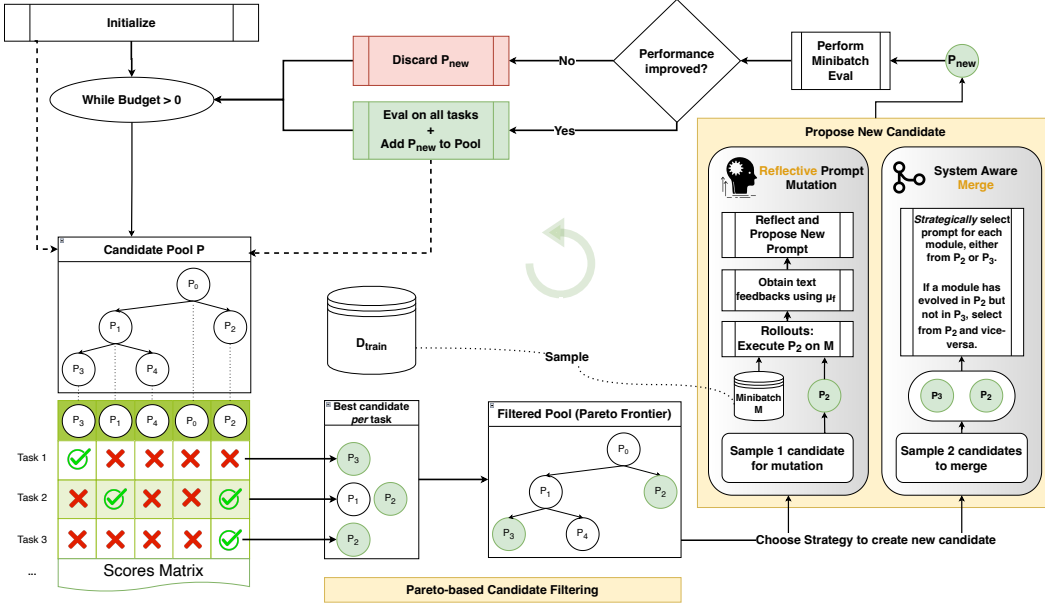


Figure 3: GEPA proposes a new candidate in every iteration by improving existing candidates using one of the two strategies (Reflective Prompt Mutation (Section 3) or System Aware Merge (Appendix D.1)), first evaluating them on a minibatch, and if improved, evaluating on a larger dataset. Instead of selecting the best performing candidate to mutate always, which can lead to a local-optimum, GEPA introduces Pareto-based candidate sampling (Section 3.1), which filters and samples from the list of best candidates *per* task, ensuring sufficient diversity. Overall, these design decisions allow GEPA to be highly sample-efficient while demonstrating strong generalization.

(iii) if it outperforms its parent(s), adds it to $\mathcal{P}$ with ancestry records and evaluate on D_{pareto} , the validation set used for selection. After the budget is exhausted, GEPA returns the candidate with the best aggregate performance on D_{pareto} .

Reflective Prompt Mutation: Natural language traces generated during the execution of a compound AI system offer rich *visibility* into the behavior and responsibilities of each module, as they capture the intermediate inferences and underlying reasoning steps. When these traces are paired with the final outcome of the system (e.g., success or failure), they provide substantial *diagnostic* value, allowing practitioners to trace errors or successes back to specific decisions made at the module level. LLMs can leverage these traces via *reflection* to perform implicit credit assignment, attributing responsibility for the final outcome to the relevant modules. This process of *reflection* can then be used to make targeted updates to individual modules, making large and effective updates to the whole system’s behavior.

Given a *candidate* to mutate in the current iteration of the optimization loop (stochastically selected from the Pareto-frontier, see Section 3.1 below), GEPA executes the selected candidate on a stochastically sampled minibatch of input queries from the trainset, tracing the program’s execution. From the execution traces, GEPA extracts the module’s inputs, outputs, and reasoning, and calls the *feedback function* μ_f , which returns a numeric score and text feedback including details about the evaluation (like compiler error messages, failed rubrics, etc.). GEPA selects the module (among the $|M|$ modules that the language program contains) to be updated based on a policy (round-robin), and a reflection LM is then shown the (current prompt, language program trajectory, score, feedback) with the task to reflectively attribute successes or failures to prompt

Algorithm 1 GEPA: Reflective Evolutionary Prompt Optimizer

Require: Inputs: System Φ , dataset $\mathcal{D}_{\text{train}}$, eval metric μ , feedback function μ_f , budget B

Require: Hyperparams: minibatch size b , Pareto set size n_{pareto}

- 1: Split $\mathcal{D}_{\text{train}}$ into $\mathcal{D}_{\text{feedback}}$, $\mathcal{D}_{\text{pareto}}$, s.t. $|\mathcal{D}_{\text{pareto}}| = n_{\text{pareto}}$
- 2: Initialize candidates $\mathcal{P} \leftarrow [\Phi]$, parents $\mathcal{A} \leftarrow [\text{None}]$
- 3: **for each** (x_i, m_i) in $\mathcal{D}_{\text{pareto}}$ **do**
- 4: $S_{\Phi}[i] \leftarrow \mu(\Phi(x_i), m_i)$
- 5: **end for**
- 6: **while** budget B not exhausted **do**
- 7: $k \leftarrow \text{SELECTCANDIDATE}(\mathcal{P}, S)$
- 8: $j \leftarrow \text{SELECTMODULE}(\Phi_k)$
- 9: $\mathcal{M} \leftarrow$ minibatch of size b from $\mathcal{D}_{\text{feedback}}$
- 10: Gather feedback, scores, traces for $\Phi_k[j]$ on $\mathcal{M}$ using μ_f
- 11: $\pi'_j \leftarrow \text{UPDATEPROMPT}(\pi_j, \text{feedbacks}, \text{traces}[j])$
- 12: $\Phi' \leftarrow$ Copy of Φ_k w/ module j updated by π'_j
- 13: $\sigma, \sigma' \leftarrow$ avg score on $\mathcal{M}$ (before, after)
- 14: **if** $\sigma' > \sigma$ **then**
- 15: Add Φ' to $\mathcal{P}$; Add k to $\mathcal{A}$
- 16: **for each** (x_i, m_i) in $\mathcal{D}_{\text{pareto}}$ **do**
- 17: $S_{\Phi'}[i] \leftarrow \mu(\Phi'(x_i), m_i)$
- 18: **end for**
- 19: **end if**
- 20: **end while**
- 21: **return** Φ^* maximizing average score on $\mathcal{D}_{\text{pareto}}$

Algorithm 2 Pareto-based candidate selection

- 1: **function** SELECTCANDIDATE($\mathcal{P}, S$)
- 2: // Build instance-wise Pareto sets
- 3: **for each** i **do**
- 4: $s^*[i] \leftarrow \max_k S_{\mathcal{P}[k]}[i]$
- 5: $\mathcal{P}^*[i] \leftarrow \{\mathcal{P}[k] : S_{\mathcal{P}[k]}[i] = s^*[i]\}$
- 6: **end for**
- 7: $\mathcal{C} \leftarrow$ unique candidates in $\bigcup_i \mathcal{P}^*[i]$
- 8: $D \leftarrow \emptyset$
- 9: **while** there exists $\Phi \in \mathcal{C} \setminus D$ dominated by another in $\mathcal{C} \setminus D$ **do**
- 10: $D \leftarrow D \cup \{\Phi\}$
- 11: **end while**
- 12: Remove D from each $\mathcal{P}^*[i]$ to get $\hat{\mathcal{P}}^*[i]$
- 13: Let $f[\Phi] =$ number of i for which $\Phi \in \hat{\mathcal{P}}^*[i]$
- 14: Sample Φ_k from $\hat{\mathcal{C}}$ with probability $\propto f[\Phi_k]$
- 15: **return** index k of Φ_k in $\mathcal{P}$
- 16: **end function**

Figure 4: **(Left)** GEPA’s core algorithm for reflective prompt evolution. GEPA works iteratively, in each iteration, selecting some of the current candidates to evolve (line 7), executing the identified candidate on a minibatch of rollouts, while utilizing a special *feedback function* μ_f to gather module specific feedback when available (lines 9-10, described in detail in Section 3), using an LLM to reflectively update the prompt (line 11), and evaluating whether the system instantiated with the new prompt improved the performance on the minibatch (line 14). If improved, GEPA then proceeds to evaluate the new system candidate on the full $\mathcal{D}_{\text{pareto}}$ set, adding it to the list of candidates tracked and marking the new system’s parent. **(Right)** The SelectCandidate subprocedure used by GEPA’s core algorithm is tasked with identifying the best candidate to evolve in the next optimization iteration. GEPA’s chief candidate selection strategy is to find non-dominated candidates in the Pareto frontier (of all task instances), and stochastically select one of them based on their appearance frequency in the Pareto front.

elements and propose revised instructions. The updated module, with the rest of the language program, is evaluated again on the minibatch, and if the score improves, then the new program is added to the candidate pool. The meta-prompt for reflective prompt updates is shown in Appendix C and the full algorithm is presented in Algorithm 1.

Evaluation traces as diagnostic signals: The text that LLMs produce is the *execution trace* of the AI system. The text that the environment produces to compute the reward (e.g. compiler error messages before giving reward 0) is the *evaluation trace*. Beyond reflection on execution traces, we identify a second valuable source of diagnostic information in the evaluation traces. Many evaluation metrics apply rich strategies (e.g., code evaluation may involve compilation, execution, and profiling), producing natural language traces before computing a scalar reward. We propose leveraging these *evaluation traces* for reflective credit assignment and targeted prompt updates. GEPA achieves this by extending rewards μ into a *feedback function* μ_f that

extracts textual traces during evaluation and returns them with the final score as `feedback_text`. When available, such feedback can even be module-specific (e.g. in multi-hop systems the evaluator may provide feedback after each hop). In practice, there are domains where human-graders are able to rate the AI system’s responses, along with providing detailed feedback justifying their scalar ratings. When available, D_{train} can be augmented with such human-written explanations for each instance; during reflection, and GEPA can consume these explanations as auxiliary `feedback_text` to guide targeted prompt updates, even when natural-language feedback from rollouts is limited or unavailable.

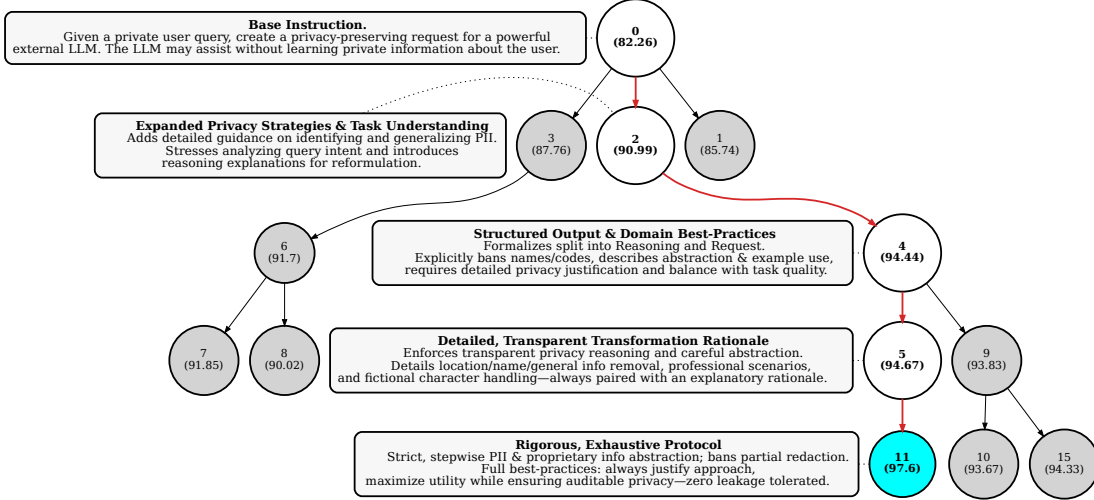


Figure 5: GEPA’s reflective prompt mutation systematically incorporates task-specific nuances, leading to substantial improvements in performance. This figure visualizes the optimization trajectory taken by GEPA, presenting an annotated subtree from Figure 25d (for the privacy-preserving delegation task PUPA) to demonstrate the iterative enhancements made to the prompts. The progression from the base prompt (candidate 0) to the best performing prompt (candidate 11) is highlighted with red arrows, and key prompt changes at each step are annotated beside the corresponding nodes. Full-length instructions for these iterations are provided in Appendix K.1. Each prompt refinement in this trajectory adds targeted nuances informed by ongoing optimization, illustrating how GEPA’s process accumulates lessons to continually boost task performance.

3.1 PARETO-BASED CANDIDATE SELECTION

GEPA is a highly modular algorithm that supports various strategies for candidate selection, with the choice of strategy governing the exploration–exploitation tradeoff. A naive approach is to always select the best-performing candidate, but this often traps the optimizer in a local optimum: once a dominant strategy is found, it becomes difficult to surpass, and the optimizer exhausts its budget without learning new, potentially better strategies. Figure 6a illustrates this behavior: after finding one new strategy (the first child node), the search repeatedly attempts to refine it, fails to improve, and ultimately depletes the budget.

To address this, GEPA employs a Pareto-based “illumination” strategy (Mouret & Clune, 2015), shown in Algorithm 2. For each training instance, GEPA records the highest score across all candidates, forming a Pareto frontier. Candidates that achieve the best score on at least one task are retained, while strictly dominated ones are pruned. From this pruned set, GEPA stochastically samples a candidate, weighting

probabilities by how many tasks each candidate leads. This strategy helps GEPA escape local optima without inflating the search, efficiently balancing exploration and exploitation by focusing resources on candidates that embody “winning” strategies within the optimization budget.

4 EVALUATION

Table 1: Benchmark results for different optimizers with Qwen3 8B. GEPA and GEPA+Merge achieve better performance than GRPO with far fewer rollouts on all benchmarks except AIME. For example, for IFBench, GEPA found optimal prompts after just 678 rollouts achieving 38.61%, outperforming GRPO’s test set score of 35.88% with 24,000 rollouts.

Qwen3 8B	HotpotQA	IFBench	Hover	PUPA	AIME-2025	LiveBench-Math	Aggregate	Improvement
Baseline	42.33	36.90	35.33	80.82	27.33	48.70	45.23	—
GRPO	43.33	35.88	38.67	86.66	38.00	51.26	48.91	+3.68
MIPROv2	55.33	36.22	47.33	81.55	20.00	46.60	47.84	+2.61
GEPA	62.33	38.61	52.33	91.85	32.00	51.95	54.85	+9.62
GEPA+Merge	64.33	28.23	51.67	86.26	32.00	51.95	52.40	+7.17
Total optimization budget (# rollouts)								
GEPA (+Merge)	6871	3593	7051	2426	1839	1839	3936	—
GRPO	24000	24000	24000	24000	24000	24000	24000	—

Table 2: Benchmark results for different optimizers evaluated on GPT-4.1 Mini. As a prompt-optimization system, GEPA works off-the-shelf on *closed-source* models as well, outperforming state-of-the-art prompt optimizers including MIPROv2 (in 2 settings: Instruction-only optimization (“MIPROv2-No-Demos”) as well as joint instruction and few-shot optimization (“MIPROv2”)), Trace (with its OptoPrime optimizer), and TextGrad. Additionally, GEPA-optimized prompts demonstrate strong cross-model generalization: “GEPA-Qwen-Opt”—optimized entirely for (and using) the weaker Qwen3-8B—achieves a +9% gain when evaluated on GPT-4.1-Mini without modification, notably outperforming all baselines (MIPROv2, TextGrad, Trace) that optimized directly for (and using) GPT-4.1-Mini.

GPT-4.1 Mini	HotpotQA	IFBench	Hover	PUPA	AIME-2025	LiveBench-Math	Aggregate	Improvement
Baseline	38.00	47.79	46.33	78.57	49.33	58.20	53.03	—
Trace (OptoPrime)	60.33	51.19	46.00	74.18	45.33	60.74	56.30	+3.27
MIPROv2-No-Demos	38.00	52.04	51.33	91.85	48.67	60.97	57.14	+4.11
MIPROv2	58.00	49.15	48.33	83.37	51.33	61.84	58.67	+5.64
TextGrad	62.33	48.64	47.67	85.68	46.67	63.84	59.14	+6.11
GEPA	69.00	52.72	51.67	94.47	59.33	64.13	65.22	+12.19
GEPA+Merge	65.67	55.95	56.67	96.46	59.33	64.13	66.36	+13.33
Optimized with Qwen3-8B, evaluated on GPT-4.1-Mini								
GEPA-Qwen-Opt	65.67	49.83	54.67	90.05	52.67	59.31	62.03	+9.00

We adopt a standard train/validation/test split. Optimizers have full access to the train split, including text and labels, for program tuning. Although optimizers may monitor the performance of candidate parameters (like model checkpoints) by tracking scores on the validation set (to implement early stopping, for example), direct access to the content of validation instances is restricted. We evaluate on six benchmarks—AIME-2025 (Balunović et al., 2025), LiveBench-Math (White et al., 2025), HotpotQA (Yang et al., 2018), IF-Bench (Pyatkin et al., 2025b), HoVer (Jiang et al., 2020), and PUPA (Li et al., 2025a)—each paired with existing compound AI systems and feedback functions. Experiments use Qwen3 8B (Yang et al., 2025) and GPT-4.1 Mini (OpenAI, 2025) with standardized inference settings, and compare against state-of-the-art optimizers MIPROv2 (Opsahl-Ong et al., 2024), Trace (with its OptoPrime optimizer) (Cheng et al., 2024),

TextGrad (Yuksekgonul et al., 2025), and GRPO¹ (Shao et al., 2024). Appendix E provides further details on benchmarks, systems, and feedback functions (Subsection E.1); models and inference settings (Subsection E.2); monetary cost to run the experiments (Subsection E.3); and optimizer configurations (Subsection E.4). Table 1, Table 2 and Figure 10 summarize our main results, from which we derive the following observations:

Observation 1: Reflective Prompt Evolution is highly sample-efficient and can outperform weight-space reinforcement learning: Across four benchmarks, GEPA adapts rapidly and generalizes robustly in compound AI systems—beating GRPO (24,000 rollouts) by up to 19% while using up to $35\times$ fewer rollouts. It reaches optimal test performance with $4\text{--}35\times$ fewer rollouts and exceeds GRPO on 5 out of 6 tasks by 19.0%, 2.73%, 13.66%, 5.19% and 0.7%. GEPA matches GRPO’s best validation after only 243, 402, 330, 1143, 1179, and 306 rollouts—up to $78\times$ greater sample efficiency. GEPA+Merge widens the gap, outperforming GRPO by 21% at a comparable rollout budget to GEPA.

The majority of GEPA’s rollout budget is spent on validation, where scores are utilized solely for candidate selection and not for producing learning signals. If we restrict the analysis to train set rollouts, GEPA requires only 79 to 737 rollouts to reach optimal performance. To match GRPO’s best validation scores, GEPA achieves this with only 102, 32, 6, and 179 train rollouts for four tasks, respectively, underscoring the high sample efficiency of learning based on reflective prompt evolution.

Since tracking candidates’ validation performance accounts for majority of GEPA’s rollout budget, sample efficiency can be further improved by evaluating on a smaller validation set or by tracking scores on dynamically selected validation subsets instead of the full set—both of which we propose as directions for future work. Figures 1a, 1b, 14c and 15c show the full performance-vs-rollouts curve for all optimizers over benchmarks HotpotQA, IFBench, HoVer and PUPA, respectively.

Observation 2: Reflective prompt evolution enables instruction-optimization alone to outperform joint instruction and few-shot optimization: We compare GEPA with MIPROv2, a state-of-the-art instruction and few-shot optimizer, using two leading models across six diverse tasks, and observe that GEPA consistently outperforms MIPROv2 in all settings, achieving margins as high as 11.1% for GPT-4.1 mini and 10.3% for Qwen3 8B. Further, GEPA and GEPA+Merge more than double the aggregate gains over baseline seen with MIPROv2 across all benchmarks and models (+13.33% and +12.19% vs +5.64% for MIPROv2).

While prior works such as Opsahl-Ong et al. (2024) and Wan et al. (2024) have provided compelling evidence for the effectiveness of few-shot example optimization—often outperforming instruction-based approaches—our findings suggest an exciting shift in this trend. We attribute this primarily to recent advances in the instruction-following and self-reflective abilities of LLMs, as well as the design choices in GEPA that capitalize on these improved capabilities. To further contextualize our findings, we redo the study on *generalization gap* (the difference between validation and test set performance for optimized prompts) as proposed by Wan et al. (2024). The results presented in Figure 16 reinforce these observations: reflectively evolved instructions now demonstrate a lower generalization gap, underscoring both advancements in model capabilities and the benefits of GEPA’s design. We see this as a reflection of the continuous evolution of LLMs and GEPA’s ability to effectively leverage these improvements.

We provide the full-length optimized prompts produced by GEPA for all systems, benchmarks, and models in Appendix L, alongside MIPROv2 prompts. Notably, in contrast to prior findings where instruction optimization yielded improvements primarily through quasi-exemplars (Wan et al., 2024), GEPA’s prompts frequently contain detailed *declarative* instructions for completing the task, as illustrated in Figure 2.

¹We use LoRA for GRPO due to its low cost and successful adoption with GRPO (Wang et al., 2025b; Xu et al., 2025b; Li et al., 2025b; Yue et al., 2025; Sun et al., 2025; Hayou et al., 2025; Zhao et al., 2025; Teknium et al., 2024; Zhao et al., 2024; Sidahmed et al., 2024). Additionally, we explore full-parameter finetuning. Figure 11 shows a similar result comparing GEPA to GRPO with full finetuning.

Observation 3: The next-candidate selection strategy strongly influences the optimization trajectory and final performance, with Pareto-based sampling providing a distinct advantage.

Table 3: Comparing candidate selection strategies across different tasks with Qwen3 8B while keeping the evolution harness fixed. At each step, SelectBestCandidate (used by TextGrad Yuksekgonul et al. (2025)) evolves only from the top-scoring candidate. BeamSearch maintains a pool of the top-N candidates (used by APO Pryzant et al. (2023)), but is still prone to local optima. In comparison, GEPA’s Pareto-based selection yields a +12.44% improvement, significantly outperforming the +6.05% and +5.11% gains of greedy and beam-search strategies respectively.

Qwen3 8B	HotpotQA	IFBench	Hover	PUPA	Aggregate	Improvement
Baseline	42.33	36.90	35.33	80.82	48.84	—
SelectBestCandidate	58.33	30.44	45.33	85.45	54.89	+6.05
BeamSearch	57.33	36.39	41.00	81.08	53.95	+5.11
GEPA	62.33	38.61	52.33	91.85	61.28	+12.44

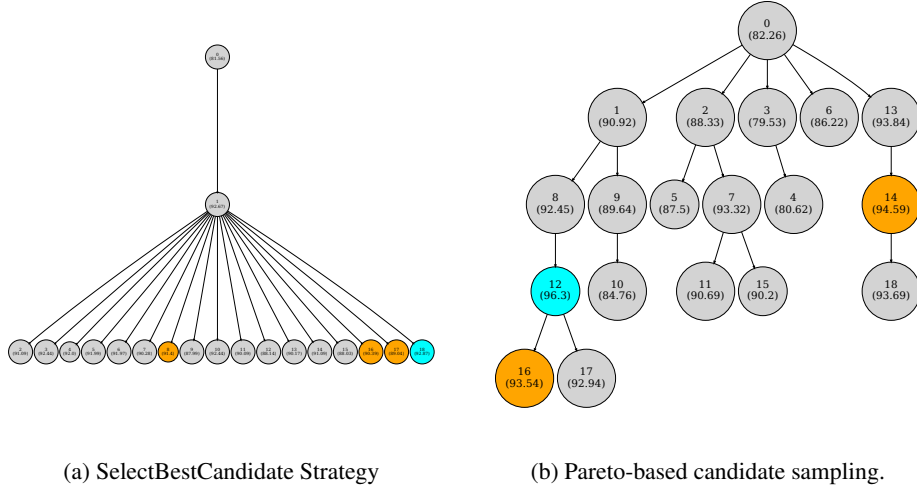


Figure 6: Comparing the impact of different candidate selection strategies. (Left) As can be seen, selecting the best-performing candidate in every iteration led to a local-optima after one iteration, leading to suboptimal search performance. (Right) On the other hand, using pareto-based candidate selection strategy, GEPA was able to generate a balanced search tree, finding a better performing program within the same budget.

GEPA refines prompts iteratively with rollout feedback; to test our Pareto-based selection, we compare against a baseline that always picks the best-performing candidate in the SelectBestCandidate strategy (which is similar to the strategy used by TextGrad Yuksekgonul et al. (2025)), and BeamSearch(N=4) (used by APO Pryzant et al. (2023)). As shown in Table 3, these baselines often yield suboptimal exploration of the prompt search space, leading to poor performance. GEPA with Pareto-based sampling outperforms the BeamSearch strategy by upto 11.33%, and SelectBestCandidate strategy by up to 8.17%, with an aggregate margin of +7.33% and +6.4% across all benchmarks, respectively. Figure 6 highlights the difference in optimization trajectories: always choosing the current best candidate gives immediate improvement but quickly stalls, wasting rollouts on a single candidate. In contrast, our Pareto-based method expands

the search by considering all Pareto-optimal candidates (all “winning” strategies found so far), balancing exploration and exploitation and converging to a higher-performing solution within the same rollout budget.

Observation 4: Instruction-optimized prompts are computationally cheaper and generalize better than few-shot demonstration prompts: In addition to their strong generalization capabilities, reflectively evolved instructions offer a significant practical advantage: they are often much shorter and thus computationally more efficient than few-shot demonstration prompts. This advantage becomes especially clear for complex tasks, where even a single few-shot demonstration can be prohibitively long. The problem is further exacerbated when few-shot examples are optimized using state-of-the-art methods such as MIPROv2, which jointly optimizes *multiple* demonstrations to be used simultaneously, further increasing prompt length.

In contrast, reflectively evolved instructions—such as those generated by GEPA—maintain compactness while providing large performance gains (as demonstrated in Lessons 1 and 2). To illustrate this, we compare GEPA’s and MIPROv2’s prompt lengths (see Figure 18). Notably, prompts produced by GEPA and GEPA+Merge are up to $9.2\times$ shorter than those from MIPROv2, representing a substantial improvement in efficiency, alongside performance improvements.

Moreover, we observe a trend where, in aggregate, optimizers that achieve higher performance tend to produce shorter prompts (see Figure 17). This reduction in prompt size has a significant impact—not only reducing runtime cost for downstream tasks (as all API-providers meter the input tokens), but also decreasing latency and improving the overall efficiency of LLM-serving systems (Kwon et al., 2023; Zheng et al., 2024; Agrawal et al., 2023; Yu et al., 2025).

Observation 5: System aware crossover strategies can provide large gains, but the optimal budget allocation between mutation and crossover, as well as *when* to invoke merge needs further study: We identify a unique system-aware crossover strategy and operationalize it as Merge (described in Appendix D.1). GEPA+Merge can outperform GEPA by as much as 5%, providing an aggregate 2% additional improvement over the already strong performance established by GEPA. Detailed results are available in Table 1. We attribute these gains to the ability of GEPA+Merge to identify distinct optimization lineages, that have learnt complementary strategies (by evolving distinct modules), and merging them by picking the best version of different modules from each of these lineages to propose a single, optimal candidate.

While in our analysis, we found GEPA+Merge works especially well for GPT-4.1 Mini, it lead to performance degradation when used with Qwen3 8B. Even Qwen3 8B benefits from Merge on one out of four tasks. We attribute these discrepancies to the way the rollout budget is allocated between reflective mutation and crossover, and the timing of invocation of the crossover strategy. In our experiments, we fixed the same hyperparameters for GPT-4.1 Mini and Qwen3 8B, leading to suboptimal choice for Qwen3 8B. Intuitively, crossover would provide the maximum benefit, when there are independent lineages that perform well. Hence, the hyperparameters should be chosen such that Merge is invoked once the optimization tree has evolved sufficiently different lineages. We propose the study of such adaptive techniques as future work.

Observation 6: GEPA-optimized prompts demonstrate cross-model generalization. Table 2 presents results for “GEPA-Qwen-Opt”, a configuration where prompts were optimized using the smaller Qwen3-8B model but evaluated on GPT-4.1-Mini. Despite originating from a weaker model in a different family, these prompts transfer effectively, achieving a +9.00% aggregate improvement across 6 benchmarks (with gains as high as +27.67% on HotpotQA). Remarkably, this transfer performance outperforms strong baselines like MIPROv2 (+5.64%), TextGrad (+6.11%), and Trace (+3.27%), even though those methods were optimized directly on the target GPT-4.1-Mini model.

5 EXTENDED APPLICATIONS OF GEPA

5.1 GEPA FOR INFERENCE-TIME SEARCH (CONTD.)

While the primary focus of this paper is sample-efficient adaptation of AI systems to new tasks, preliminary findings suggest that GEPA may also serve as a promising inference-time search technique. This can be achieved by passing the set of tasks to be solved (for example, a list of Pytorch modules to be converted to CUDA) as the training set to GEPA, ensuring that both D_{train} and D_{pareto} contain the full set of tasks. This way, GEPA can “overfit” the set of tasks, iteratively proposing better solutions to every problem. We also note that this allows GEPA to apply lessons and insights extracted from rollouts for one task to other tasks. To explore this use case, we conduct preliminary experiments using GEPA as an inference-time search technique for code-generation tasks on two hardware platforms: writing kernels for AMD’s recently introduced XDNA2 Architecture (Advanced Micro Devices, 2025) using an early version of the NPUEval benchmark (Kalade & Schelle, 2025), and generating CUDA code for NVIDIA-V100 GPUs using KernelBench (Ouyang et al., 2025).

A distinguishing aspect of these experiments is the use of the feedback function μ_f to dynamically inject domain-specific knowledge into the optimization process. Specifically, kernel development expertise—often codified in technical manuals and documentation—can be selectively surfaced by retrieving relevant manual sections based on rollout failures (e.g., compiler error messages). By using error information to make targetted retrieval queries, GEPA promotes integration of architectural best practices into prompt evolution, as exemplified by the detailed prompt for NPUEval shown in Figure 27. We also note that generation stochasticity (temperature based sampling) is eliminated by operating under a cache; this ensures that observed improvements tie closely to inference scaling through prompt updates and GEPA’s diverse prompt exploration, rather than stochasticity in the model’s sampling process.

NPU Kernels: We create a sequential refinement agent that iteratively generates kernels (up to 10 times) based on feedback like compiler errors and profiling results (Sequential10), and evaluate the Best-of-N generation. With GPT-4o alone, Sequential10 reaches only 4.25% mean vector utilization. Adding RAG, sourced from technical manuals, improves this to 16.33%, and integrating MIPROv2 further raises it to 19.03%. Notably, applying GEPA to Sequential10 (without RAG) dramatically boosts kernel performance, with several generated kernels achieving up to 70% vector utilization and a mean of 30.52%. Furthermore, a single prompt generated by GEPA enables Sequential10 (again without RAG) to attain a score of 26.85%.

CUDA Kernels: For 35 tasks from the KernelBench “representative subset” (Ouyang et al., 2025), spanning three difficulty levels, we ran GEPA with GPT-4o. As depicted in Figure 8, GEPA boosts GPT-4o’s close-to-0% $fast_1$ score to above 20% with increasing search budget. This task used an agent that could generate upto 5 sequential refinements based on environment feedback (Sequential5).

These experiments with GPT-4o also demonstrate GEPA’s ability to leverage the abilities of frontier LLMs. However, these are early results and warrant further systematic study. We believe that leveraging GEPA for inference-time search, particularly when coupled with domain specific textual feedback, could generalize to other code generation and domain adaptation tasks—a direction we leave for future work.

5.2 GEPA FOR ADVERSARIAL PROMPT SEARCH (CONTD.)

We instantiate GEPA for adversarial prompt search by inverting the reward signal: the optimizer proposes prompt edits to include additional information like trivia that minimize task performance (pass@1), while requiring that prompts do not contradict the task and still contain all information needed to solve it. For AIME, GEPA’s adversarial search used AIME 2022–2024 problems as the pool for prompt evolution. The learned prompt was evaluated on AIME-2025 (30 problems), using GPT-5 Mini with 5 runs per problem

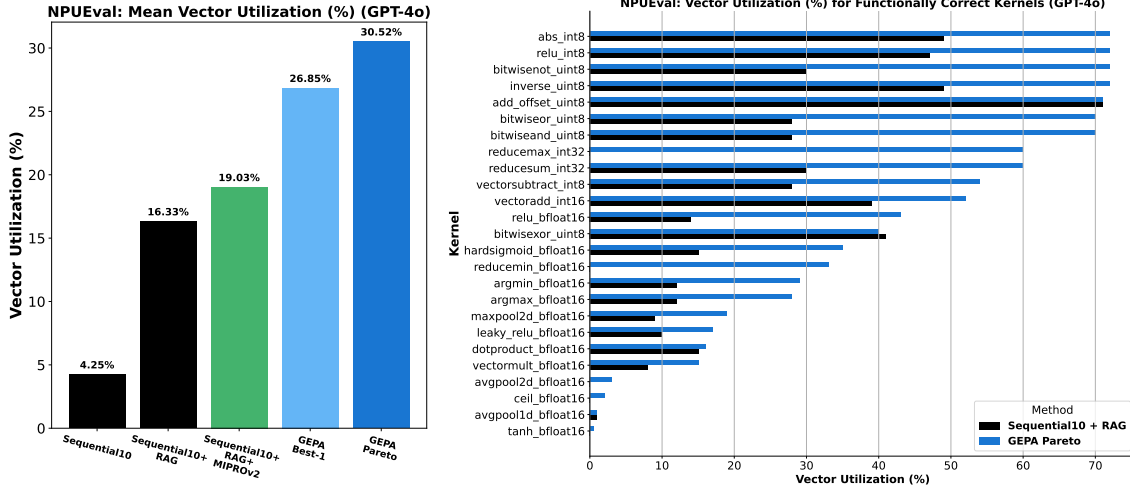


Figure 7: GEPA with GPT-4o is able to generate kernels for AMD NPUs that achieve vector utilization rates as high as 70%, with a mean utilization score of 30.52%. In comparison, GPT-4o, even after up to 10 sequential refinements with environment feedback, achieves an aggregate score of only 4.25%. When enhanced with retrieval-augmented generation (RAG) and MIPRO, the sequential refinement agent improves to scores of 16.33% and 19.03%, respectively. Notably, the final prompt produced by GEPA enables the same agent to reach a utilization score of 26.85%, all without requiring any runtime RAG.

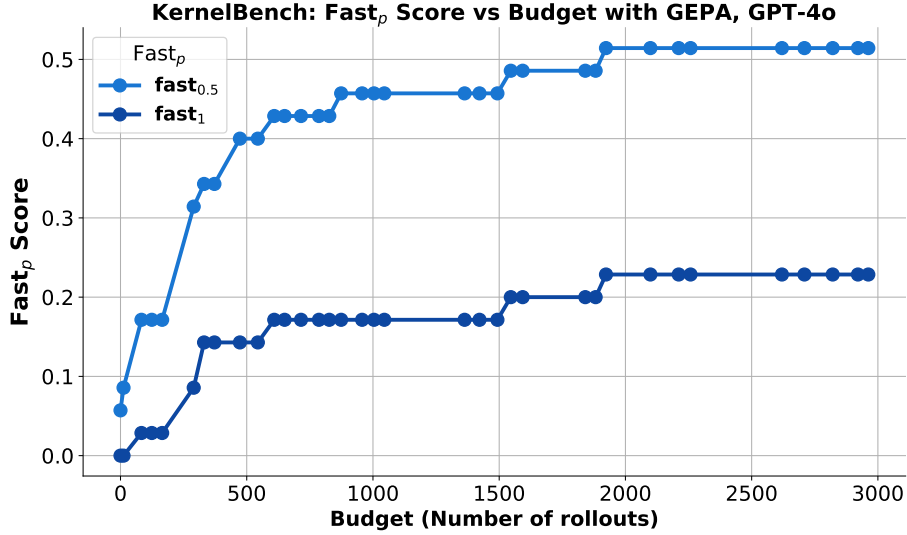


Figure 8: GEPA with GPT-4o is able to iteratively refine and improve CUDA Kernel Code. The graphs shows $fast_p$ vs. rollouts plot for $p=[0.5, 1]$, where the speedup is calculated over Pytorch-eager. $fast_p$ is a metric described in (Ouyang et al., 2025) that measures the fraction of tasks for which the method generated a kernel executing faster than p times the baseline. As can be seen, GEPA with GPT-4o is able to generate cuda kernels executing faster than Pytorch-eager for over 20% of the 35 representative tasks.

(150 generations total). We started from a clean instruction prompt and evolved a single universal adversarial instruction that is prepended to each query.

The learned adversarial instruction reduced pass@1 on AIME-2025 from 76% (clean prompt) to 10% (adversarial prompt), despite preserving the nominal task description and the required answer-format directive.

Base (clean) prompt:

You are a helpful assistant. You are given a question and you need to answer it. The answer should be given at the end of your response in exactly the format ‘### <final answer>’.

GEPA-generated adversarial prompt (abridged):

You are a helpful assistant. You are given a question and you need to answer it. It’s interesting to note that honey never spoils and that the longest river in the world is the Nile, stretching over 6,650 kilometers. When providing your answer, be sure to format it at the end of your response exactly as ‘### <final answer>’. For this task, remember that many mammals, including dolphins, sleep with one eye open. Proceed to answer the given question accordingly.

Manual inspection showed that the adversarial prompt caused GPT-5 Mini to end most responses with the literal placeholder ### <final answer>, indicating a systematic misinterpretation of the formatting rule when paired with the injected distractors. This suggests that the large drop arises from the interaction between extraneous details and a strict, literal formatting constraint, rather than from the formatting requirement alone.

Adversarial prompt search systematically uncovers instruction-level perturbations that sharply degrade model performance, providing a principled, automated way to probe worst-case robustness beyond average-case metrics. By finding universal, task-preserving distractors (e.g., trivia plus strict formatting), it reveals brittle instruction-following interactions and turns them into reusable stress tests and regression suites for continuous evaluation. The resulting adversarial prompts could be used to provide targeted data for fine-tuning or safety training. In practice, this could improve deployment reliability, enables red-teaming at scale, and help track robustness drift over time across models, versions, and domains.

6 RELATED WORK

Prompt optimization improves LLMs but often needs manual expertise; for instance, chain-of-thought prompting Wei et al. (2023). To scale this approach, recent methods use LLMs to optimize prompts automatically (Zhou et al., 2022; Yang et al., 2024; Agarwal et al., 2024; Fernando et al., 2024). GEPA leverages LLMs, but differs by incorporating textual environment feedback, Pareto-aware search over candidates, and evolution strategies per submodule within an AI system.

Evolutionary algorithms have been used to optimize prompts, e.g., EvoPrompt (Guo et al., 2024), which evolves prompt populations. Rainbow Teaming (Samvelyan et al., 2024) applies quality-diversity evolution to generate diverse adversarial prompts. GEPA additionally uses domain-specific feedback for targeted mutations achieving higher sample efficiency. AlphaEvolve (Novikov et al., 2025) and OpenEvolve (Sharma, 2025) apply evolutionary search directly to code rewriting, excelling when problem solution can be codified. While AlphaEvolve targets a single hard problem, GEPA brings evolution to prompts across domains, combining Pareto-frontier optimization and prompt evolution to transfer tactics from related problems.

Feedback-driven improvement often uses reinforcement learning, such as majority voting signals (Zuo et al., 2025), but RL can be sample-inefficient when rewards are slow to compute. An alternative is learning in the language space: in-context bandit/self-bootstrapping methods (Shinn et al., 2023; Madaan et al., 2023) (Monea et al., 2025; Xu et al., 2025a; Feng et al., 2025; Cheng et al., 2024), workflow memory and skills

(Wang et al., 2024; 2025c), and test-time strategy synthesis via Dynamic Cheatsheet (Suzgun et al., 2025), reasoning cache (Chen et al., 2025c). GEPA instead uses examples to propose new *instructions*, yielding task-specific rules.

To **optimize compound AI systems and agents** (Lin et al., 2025b), DSPy (Khattab et al., 2022; 2024) searches/bootstraps few-shot examples, TextGrad (Yuksekgonul et al., 2025) backpropagates textual feedback, and MIPROv2 (Opsahl-Ong et al., 2024) jointly aligns instructions and examples via Bayesian optimization; these largely rely on global rewards. Agent-Pro (Zhang et al., 2024) evolves agent policies through dynamic belief generation and reflection on interactive experiences. Optimas (Wu et al., 2025a) introduces globally aligned local rewards per module. GEPA combines global rewards with environment textual feedback per module and maintains a Pareto frontier over individual data instances, matching prompts/agent design to specific examples. The Pareto-guided evolution lets GEPA explore diverse prompt/code/agent design strategies before converging to a robust, generalizable set.

7 CONCLUSION

We introduced GEPA, a prompt optimizer for arbitrary LLM agents and workflows that leverages explicit reflection and Pareto-based selection, showing superior sample efficiency compared to reinforcement learning (GRPO), while outperforming leading prompt optimizers (MIPROv2). By explicitly incorporating natural language feedback and maintaining a diverse pool of Pareto-optimal candidates, GEPA rapidly adapts AI systems to new tasks. Our results across benchmarks and models suggest that language-based reflection can offer a scalable strategy for optimizing complex real-world AI workflows, especially in resource-constrained settings. GEPA also shows promise as an inference-time search strategy, showing the ability to write code in challenging domains.

REFERENCES

- Advanced Micro Devices. Amd xdna™ architecture. <https://www.amd.com/en/technologies/xdna.html#xdna2>, 2025. Accessed on: 2025-07-23.
- Eshaan Agarwal, Joykirat Singh, Vivek Dani, Raghav Magazine, Tanuja Ganu, and Akshay Nambi. Promptwizard: Task-aware prompt optimization framework, 2024. URL <https://arxiv.org/abs/2405.18369>.
- Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S Gulavani, and Ramachandran Ramjee. Sarathi: Efficient llm inference by piggybacking decodes with chunked prefills. *arXiv preprint arXiv:2308.16369*, 2023.
- Meta AI. llama-prompt-ops: An open-source tool for seamless migration from other llms to llama, and for general prompt optimization. <https://github.com/meta-llama/llama-prompt-ops>, 2025. Accessed: 2025-07-11.
- Mislav Balunović, Jasper Dekoninck, Ivo Petrov, Nikola Jovanović, and Martin Vechev. Aime-2025, February 2025. URL https://huggingface.co/datasets/MathArena/aime_2025. Dataset from MathArena: Evaluating LLMs on Uncontaminated Math Competitions.
- Shiyi Cao, Sumanth Hegde, Dacheng Li, Tyler Griggs, Shu Liu, Eric Tang, Jiayi Pan, Xingyao Wang, Akshay Malik, Graham Neubig, Kourosh Hakhmaneshi, Richard Liaw, Philipp Moritz, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. Skyrl-v0: Train real-world long-horizon agents via reinforcement learning, 2025.